Iteration II — Common Loop Patterns

Lecture 23

CS 1001 Kirk Duran

Today

- Recognize common patterns built from `for` loops.
- Use accumulators to compute totals and counts.
- Build new lists while preserving the original input.
- Select values conditionally during iteration.
- Choose between `for` and `while` loops.
- Trace `while` loops and spot termination and boundary bugs.

Key idea

Most useful loops are not arbitrary. They are instances of a small number of recurring computational patterns.

Part 1: From Individual Loops to Patterns

A Pattern Has Four Parts

Many loops can be understood using four questions.

1. **Initialization:** What state exists before the loop begins?
2. **Iteration:** What values are examined?
3. **Update:** How does the state change during each iteration?
4. **Result:** What value is used after the loop finishes?

Example questions

Is the state an integer count? A running total? A new list being constructed?

Same Loop Structure, Different Computation

Running total

```
total = 0
for value in values:
   total = total + value
```

Counting matches

```
count = 0
for value in values:
   if value > 0:
      count = count + 1
```

The control structure is similar. The **meaning of the changing state** is different.

Part 2: Accumulation and Counting

Pattern 1: Accumulating a Total

Function

```
def tally(nums:  list[int]) -> int:
   total = 0
   for num in nums:
      total = total + num
   return total
```

Role	Code
initialize	`total = 0`
iterate	`for num in nums`
update	`total = total + num`
result	`return total`

Trace the Running Total

For `tally([4, 9, 2, 1])`:

Iteration	`num`	`total` **before**	`total` **after**
1	4	0	4
2	9	4	13
3	2	13	15
4	1	15	16

Key idea

The value computed during one iteration becomes part of the starting state for the next iteration.

Pattern 2: Counting

Counting is another accumulator pattern.

Count positive values

```
count = 0
for value in values:
   if value > 0:
      count = count + 1
```

The accumulator does not store the values themselves. It stores **how many** values satisfy a condition.

Conditional Updates

Suppose:

Code

```
values = [-2, 5, 0, 7]
count = 0
for value in values:
   if value > 0:
      count = count + 1
```

Iteration	value	`value > 0`	count after
1	-2	False	0
2	5	True	1
3	0	False	1
4	7	True	2

Counting Characters in a String

The same pattern works with strings.

Function

```
def count_a(text:  str) -> int:
   count = 0
   for char in text:
      if char == "a":
         count = count + 1
   return count
```

`count_a("banana")` returns 3.

Trace `count_a("banana")`

Iteration	char	`char == "a"`	count after
1	b	False	0
2	a	True	1
3	n	False	1
4	a	True	2
5	n	False	2
6	a	True	3

The pattern is unchanged even though the sequence type is different.

Accumulator vs. Loop Variable

These names have different roles.

Name	Role
`char`	rebound to the next sequence element
`count`	stores state accumulated across iterations

Common confusion

The loop variable normally changes because the loop rebinds it. The accumulator changes because the loop body explicitly assigns a new value to it.

Bug: Initializing Inside the Loop

Incorrect

```
for value in values:
   total = 0
   total = total + value
```

Correct

```
total = 0
for value in values:
   total = total + value
```

Key idea

If the accumulator is reinitialized during every iteration, earlier progress is discarded.

Bug: Returning Too Early

Incorrect

```
def tally(nums:  list[int]) -> int:
   total = 0
   for num in nums:
      total = total + num
      return total
```

The first `return` ends the function call immediately.

Consequence

For a non-empty input, only the first element is processed.

Part 3: Building New Lists

Pattern 3: Build a New List

Some loops accumulate a **list** rather than a number.

General shape

```
result = []
for value in values:
   result.append(...)
return result
```

Key idea

The new list is the accumulator. Each iteration intentionally mutates that result list by appending another element.

Copying with a Loop

Function

```
def copy(nums:  list[int]) -> list[int]:
   new_list = []
   for idx in range(len(nums)):
      new_list.append(nums[idx])
   return new_list
```

The input is read. The new list is mutated.

`new_list is nums` is `False`, even when their contents are equal.

Trace the New List

For `copy([4, 9, 2, 1])`:

Iteration	idx	`nums[idx]`	new_list after
1	0	4	`[4]`
2	1	9	`[4, 9]`
3	2	2	`[4, 9, 2]`
4	3	1	`[4, 9, 2, 1]`

The result grows one element at a time.

Intentional Mutation vs. Unintended Mutation

In a list-building function, this is intentional:

Intentional

```
new_list = []
new_list.append(value)
```

This would change the caller's list instead:

Different behavior

```
new_list = nums
new_list.append(value)
```

Key idea

The critical design question is *which list is supposed to change?*

Pattern 4: Transform Every Value

A common pattern is to produce one new value for every input value.

Function

```
def increment_all(nums:  list[int]) -> list[int]:
   new_list = []
   for value in nums:
      new_list.append(value + 1)
   return new_list
```

Input and output have the same number of elements, but corresponding values differ.

Trace the Transformation

For `increment_all([4, 9, 2, 1])`:

Iteration	value	value + 1	new_list after
1	4	5	`[5]`
2	9	10	`[5, 10]`
3	2	3	`[5, 10, 3]`
4	1	2	`[5, 10, 3, 2]`

Part 4: Selecting Values During Iteration

Pattern 5: Build a List Conditionally

Sometimes only some input values belong in the result.

Positive values

```
result = []
for value in nums:
   if value > 0:
      result.append(value)
```

The loop examines every value. The conditional decides whether the result list changes.

Trace Conditional List Building

For `[-3, 5, 0, -1, 8, 2]`:

Iteration	**value**	`value > 0`	**result after**
1	-3	False	`[]`
2	5	True	`[5]`
3	0	False	`[5]`
4	-1	False	`[5]`
5	8	True	`[5, 8]`
6	2	True	`[5, 8, 2]`

Counting vs. Collecting

These patterns may use the same condition but produce different results.

Count matches

```
count = 0
for value in nums:
   if value > 0:
      count += 1
```

Collect matches

```
result = []
for value in nums:
   if value > 0:
      result.append(value)
```

One result is an integer. The other is a list.

Transforming Only Selected Values

We can combine a condition with a transformation.

Code

```
result = []
for value in values:
   if value > 0:
      result.append(value * 2)
```

Example:

$$\texttt{[-1, 3, 0, 5]} \rightarrow \texttt{[6, 10]}$$

Key idea

The condition determines *which* values contribute. The expression determines *what* each contribution becomes.

Trace Two Accumulators at Once

For `values = [-1, 3, 0, 5]`:

Iter.	value	positive?	total	output
1	-1	False	0	`[]`
2	3	True	3	`[6]`
3	0	False	3	`[6]`
4	5	True	8	`[6, 10]`

In class

Which values determine control flow? Which variables preserve state across iterations?

Part 5: While Loops and Their Pitfalls

A While Loop Repeats While a Condition Is True

A `for` loop takes successive items from an iterable. A `while` loop tests a condition before each iteration.

General shape

```
initialize state
while condition:
   do work
   update state
```

Countdown

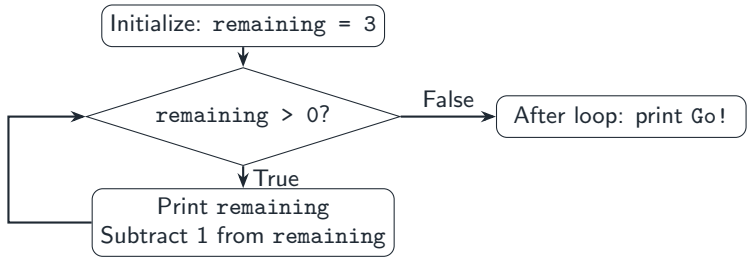
```
remaining = 3
while remaining > 0:
   print(remaining)
   remaining = remaining - 1
print("Go!")
```

Output: 3, 2, 1, then `Go!`, each on its own line.

Key idea

The condition controls repetition. The body must make progress toward stopping when termination is intended.

Follow the Condition Back Around the Loop



Key idea

Test before the first iteration and again after each complete body. An initially false condition means zero iterations.

Trace the Final False Test Too

For the countdown starting at `remaining = 3`:

Test	remaining	remaining ¿ 0	Body output	After body
1	3	True	3	2
2	2	True	2	1
3	1	True	1	0
4	0	False	none; exit loop	0

Four condition tests, three body executions. After the loop, `print("Go!")` runs once.

In class

What if `remaining` starts at `0`? The body never runs; the program prints only `Go!`.

For vs. While: Choose by What Controls Repetition

For: consume items

```
for step in range(3):
   print(step)
```

Prefer this when visiting items in a list or string, or repeating over a range. Python advances the iterator for you.

While: test a condition

```
step = 0
while step < 3:
   print(step)
   step = step + 1
```

Prefer this when stopping depends on changing state, such as a user's next response. Manage progress explicitly.

Both examples print 0, 1, 2.

Key idea

These forms can express the same computation. Choose the one that makes the stopping rule clearest; a `for` loop need not know the item count in advance.

Use While When the User Decides When to Stop

Read commands until the user types exactly `quit`:

Sentinel-controlled input

```
command = input("Command (quit to stop):  ")
while command != "quit":
   print("You entered:", command)
   command = input("Command (quit to stop):  ")
print("Done")
```

- `quit` is the **sentinel**: a special value meaning stop.
- The first input initializes the state; the input inside the body refreshes it.

Key idea

Future input determines when this stops. Without another input call, the condition keeps checking the old command.

Pitfall: No Progress Toward a False Condition

Both examples below keep the condition true indefinitely:

Missing update

```
remaining = 3
while remaining > 0:
    print(remaining)
```

`remaining` stays at 3.

Wrong direction

```
remaining = 3
while remaining > 0:
    remaining = remaining + 1
```

`remaining` grows: 4, 5, 6, ...

Fix the countdown with `remaining = remaining - 1` inside the loop.

Key idea

Check every path through the body: does the state change so the condition can eventually become false? If a loop is stuck, interrupt it with Stop or Ctrl+C, then inspect the condition and updates.

Pitfall: A Conditional Update Can Get Stuck

Incorrect

```
i = 0
while i < 3:
   if i > 0:
      print(i)
      i = i + 1
```

Correct

```
i = 0
while i < 3:
   if i > 0:
      print(i)
   i = i + 1
```

On the left, `i > 0` is false at `i = 0`, so neither statement runs. The next iteration starts with `i = 0` again.

Key idea

Indentation controls progress too. The corrected loop advances on every iteration and prints 1, then 2.

Pitfall: One Iteration Too Many

`values = [4, 9, 2]` has valid indexes 0, 1, 2.

Incorrect boundary

```
i = 0
while i <= len(values):
   print(values[i])
   i = i + 1
```

Correct boundary

```
i = 0
while i < len(values):
   print(values[i])
   i = i + 1
```

The left loop eventually tries `values[3]` and raises `IndexError`. The right loop stops before that access; it also handles an empty list.

Key idea

Trace the first and last legal iterations. For simply visiting these values, prefer `for value in values:` and avoid managing an index.

Part 6: Reasoning About Loop Functions

A General Trace Strategy

For a loop inside a function:

1. bind arguments to parameters;
2. execute initialization before the loop;
3. identify the next item (`for`) or test the condition (`while`);
4. trace one iteration at a time;
5. record every state update;
6. repeat until items are exhausted or the condition is false;
7. evaluate the final `return`.

Key idea

Do not mentally skip from the input directly to the answer. Trace the changing state.

Predict the Pattern Before Tracing

Function

```
def mystery(values:  list[int]) -> int:
   answer = 0
   for value in values:
      if value < 0:
         answer = answer + 1
   return answer
```

In class

Before calculating any particular input, classify what the function is doing. What does `answer` represent?

Prediction: Reveal

`answer` is a **count** of negative values.
For:

Call

```
mystery([3, -2, -8, 0, 5])
```

the returned value is:

2

Key idea

Recognizing the pattern often lets you understand the function before tracing every statement.

Prediction Challenge: Build a Result

Function

```
def mystery(values:  list[int]) -> list[int]:
   answer = []
   for value in values:
      if value % 2 == 0:
         answer.append(value + 10)
   return answer
```

In class

Predict `mystery([3, 4, 8, 9])`. Identify both the selection rule and the transformation.

Prediction Challenge: Reveal

The condition selects even values:

$$4, 8$$

The append expression transforms them:

$$4 + 10 = 14, \qquad 8 + 10 = 18$$

Therefore:

Result

```
[14, 18]
```

Common Bug: Mutating the Input by Accident

Suppose the specification says to return a new list.

Problematic start

```
result = values
```

Then `result` and `values` are aliases.

Later

```
result.append(10)
```

changes the caller's list too.

Safer pattern for a new result

Start with a genuinely new list such as `result = []` and append the desired values.

Common Bug: Updating the Wrong Variable

Intended count

```
count = 0
for value in values:
   if value > 0:
      value = value + 1
```

The loop executes, but the accumulator never changes.

Key idea

When debugging a loop, identify which variable is supposed to represent the result-so-far and verify that the body actually updates it.

Common Bug: Wrong Indentation

Conditional append

```
for value in values:
   if value > 0:
      result.append(value)
```

Unconditional append

```
for value in values:
   if value > 0:
      count += 1
   result.append(value)
```

Indentation determines which statements are controlled by the conditional.

Pattern Recognition Table

Goal	Initial state	Typical update
total values	0	add value
count matches	0	add 1 conditionally
transform values	[]	append transformed value
select values	[]	append conditionally
select + transform	[]	conditional transformed append

These are not separate Python features. They are recurring organizations of familiar statements.

The Same Patterns Can Be Expressed More Compactly

Once the explicit loop pattern is understood, Python provides higher-level sequence syntax for several of these operations.

$$\text{explicit loop pattern} \rightarrow \text{higher-level sequence expression}$$

Next lecture

Lecture 24 will name these sequence patterns more formally and show how list comprehensions express transformation and selection without manually constructing the loop body in the same way.

Where We Stop Today

By the end of Lecture 23, you should be able to:

- trace an accumulator across iterations;
- distinguish counting from totaling;
- build a new list without modifying the input;
- conditionally add values to a result;
- combine a condition with a transformation;
- choose between `for` and `while`, and explain termination;
- spot infinite loops, missed updates, and off-by-one errors.

Key idea

The important skill is not memorizing code fragments. It is recognizing the state a repeated computation must preserve and update.

Takeaways

1. Loop patterns are defined by their changing state.
2. Numeric accumulators support totals and counts.
3. Lists can serve as accumulators when constructing new sequential data.
4. Conditionals determine which iterations update the result.
5. Intentional mutation of a fresh output list is different from accidentally mutating an input list.
6. Use `for` to visit items and `while` for a changing condition.
7. A `while` loop needs a correct boundary and progress toward stopping.